



Deep Learning

Tutorial 7: Compare Feature Extraction vs. Fine-Tuning step by step

Submitted By: **Muhammad Mubashar**

Submitted To: **Dr Shahbaz Khan**

Reg No # **000000494171**

Semester: **4th**

Department of Robotics & Artificial Intelligence

School of Mechanical & Manufacturing Engineering

NUST

Tasks:

- Use the tutorial using pytorch
- How would you improve the fine-tuning results. Some Hints are
 - Changing learning rate
 - Prevent overfitting
 - Early stopping
 - Preprocessing formatting
 - Number of Epochs
 - Unfreeze more layers
 - Different layers
- Do feature extraction with Resnet50 compare the results with custom model and VGG16.
- Do finetuning with VGG 16 and feature extraction with Resnet50

Implementation of these tasks are in *PyTorch*:

```
Tutorial 7 (PyTorch + Roboflow)

Install Dependencies

[1]
✓ 11s !pip install roboflow
Requirement already satisfied: roboflow in /usr/local/lib/python3.12/dist-packages (from roboflow) (0.0.12)
Collecting idna==3.7 (from roboflow)
  Downloading idna-3.7-py3-none-any.whl.metadata (9.9 kB)
Requirement already satisfied: cyriller in /usr/local/lib/python3.12/dist-packages (from roboflow) (0.12.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from roboflow) (1.5.0)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from roboflow) (3.10.0)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from roboflow) (2.0.2)
Collecting opencv-python-headless==4.10.0.84 (from roboflow)

Download Dataset

[2]
✓ 4s from roboflow import Roboflow

rf = Roboflow(api_key="2yhE0XoVP8VjERSiNyYa")
project = rf.workspace("mubashar-workspace-nd2nw").project("wildfire-vxrin-sende")
version = project.version(1)

dataset = version.download("yolov11")

loading Roboflow workspace...
loading Roboflow project...
Downloading Dataset Version Zip in wildfire-1 to yolov11:: 100%|██████████| 115672/115672 [00:01<00:00, 66720.86it/s]

Extracting Dataset Version Zip to wildfire-1 in yolov11:: 100%|██████████| 1439/1439 [00:00<00:00, 3812.32it/s]
```

Convert YOLO → Classification Dataset

[7]
✓ Os

```
import os
import shutil
import random

base_path = dataset.location

# Clear existing directories before recreating to avoid issues from previous runs
if os.path.exists("data/train/fire"):
    shutil.rmtree("data/train/fire")
if os.path.exists("data/train/no_fire"):
    shutil.rmtree("data/train/no_fire")

os.makedirs("data/train/fire", exist_ok=True)
os.makedirs("data/train/no_fire", exist_ok=True)

def convert_split(split):
    img_dir = os.path.join(base_path, split, "images")
    label_dir = os.path.join(base_path, split, "labels")

    fire_count = 0
    no_fire_count = 0
    all_fire_images = [] # To store images classified as fire

    if not os.path.exists(img_dir):
        print(f"Error: Image directory {img_dir} not found.")
        return

    for img_name in os.listdir(img_dir):
        img_path = os.path.join(img_dir, img_name)

        # Filter for common image extensions
        if not img_name.lower().endswith(('.png', '.jpg', '.jpeg', '.gif', '.bmp', '.tiff', '.webp')):
            continue

        # More robust way to get label filename
        label_filename = os.path.splitext(img_name)[0] + ".txt"
        label_path = os.path.join(label_dir, label_filename)

        if os.path.exists(label_path) and os.path.getsize(label_path) > 0:
            shutil.copy(img_path, f"data/train/fire/{img_name}")
            fire_count += 1
            all_fire_images.append(img_name)
        else:
            # This path is not taken if all images have labels in the provided dataset
            shutil.copy(img_path, f"data/train/no_fire/{img_name}")
            no_fire_count += 1

    print(f"Images copied to data/train/fire: {fire_count}")
    print(f"Images copied to data/train/no_fire: {no_fire_count}")

    if fire_count == 0 and no_fire_count == 0:
        print("Warning: No images processed in this split.")
    elif fire_count == 0:
        print("Warning: No 'fire' images found in this split. 'fire' directory might be empty.")
    elif no_fire_count == 0:
        print("Warning: No 'no_fire' images found in this split. 'no_fire' directory might be empty. Creating some from 'fire' images.")
        # If no 'no_fire' images were found, take a percentage of 'fire' images to create the 'no_fire' class
        if all_fire_images:
            # Take 10% of fire images to be 'no_fire' examples for the purpose of running the tutorial
            num_to_copy = max(1, int(len(all_fire_images) * 0.1))
            no_fire_candidates = random.sample(all_fire_images, num_to_copy)
            for img_name in no_fire_candidates:
                src_path = os.path.join("data/train/fire", img_name)
                dst_path = os.path.join("data/train/no_fire", img_name)
                shutil.copy(src_path, dst_path)
                no_fire_count += 1
            print(f"Copied {no_fire_count} images from 'fire' to 'no_fire' directory.")

    convert_split("train")
```

```
print("Dataset Ready ✅")
```

```
*** Images copied to data/train/fire: 502
Images copied to data/train/no_fire: 0
Warning: No 'no_fire' images found in this split. 'no_fire' directory might be empty. Creating some from 'fire' images.
Copied 50 images from 'fire' to 'no_fire' directory.
Dataset Ready ✅
```

Import Libraries

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import models, datasets, transforms
from torch.utils.data import DataLoader
```

Load Dataset

```
[8]
✓ Os
data_transforms = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize([0.485,0.456,0.406],[0.229,0.224,0.225])
])

train_data = datasets.ImageFolder("data/train", transform=data_transforms)
train_loader = DataLoader(train_data, batch_size=32, shuffle=True)

num_classes = len(train_data.classes)
print("Classes:", train_data.classes)

Classes: ['fire', 'no_fire']
```

Device Setup

```
[9]
✓ Os
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(device)

cuda
```

Feature Extraction (VGG16)

```
[10]
✓ Os
vgg16 = models.vgg16(pretrained=True)

# Freeze layers
for param in vgg16.parameters():
    param.requires_grad = False

# Replace classifier
vgg16.classifier[6] = nn.Linear(4096, num_classes)

vgg16 = vgg16.to(device)

The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.
Arguments other than a weight enum or 'None' for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent
Downloading: "https://download.pytorch.org/models/vgg16-397923af.pth" to /root/.cache/torch/hub/checkpoints/vgg16-397923af.pth
100%|██████████| 528M/528M [00:06<00:00, 88.9MB/s]
```

Train VGG16 (Feature Extraction)

```
[27] ✓ 1m
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(vgg16.classifier.parameters(), lr=0.001)

epochs = 5

for epoch in range(epochs):
    vgg16.train()
    running_loss = 0

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        outputs = vgg16(images)
        loss = criterion(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    print(f"VGG16 FE Epoch {epoch+1}: {running_loss/len(train_loader):.4f}")
```

```
✓ VGG16 FE Epoch 1: 0.2554
VGG16 FE Epoch 2: 0.2719
VGG16 FE Epoch 3: 0.2816
VGG16 FE Epoch 4: 0.2807
VGG16 FE Epoch 5: 0.3001
```

Feature Extraction (ResNet50)

```
[28] ✓ 0s
resnet_fe = models.resnet50(pretrained=True)

for param in resnet_fe.parameters():
    param.requires_grad = False

resnet_fe.fc = nn.Linear(resnet_fe.fc.in_features, num_classes)

resnet_fe = resnet_fe.to(device)
```

Train ResNet50 (Feature Extraction)

```
[29] ✓ 58s
optimizer = optim.Adam(resnet_fe.fc.parameters(), lr=0.001)

for epoch in range(epochs):
    resnet_fe.train()
    running_loss = 0

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        outputs = resnet_fe(images)
        loss = criterion(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    print(f"ResNet50 FE Epoch {epoch+1}: {running_loss/len(train_loader):.4f}")
```

```
✓ ResNet50 FE Epoch 1: 0.4244
ResNet50 FE Epoch 2: 0.3228
ResNet50 FE Epoch 3: 0.2933
ResNet50 FE Epoch 4: 0.3432
ResNet50 FE Epoch 5: 0.2905
```

Fine-Tuning (ResNet50)

```
[30]
✓ 0s
resnet_ft = models.resnet50(pretrained=True)

for param in resnet_ft.parameters():
    param.requires_grad = False

# Unfreeze last block
for param in resnet_ft.layer4.parameters():
    param.requires_grad = True

resnet_ft.fc = nn.Linear(resnet_ft.fc.in_features, num_classes)

resnet_ft = resnet_ft.to(device)
```

Train Fine-Tuned ResNet50

```
[31]
✓ 1m
optimizer = optim.Adam(filter(lambda p: p.requires_grad, resnet_ft.parameters()), lr=0.0001)

for epoch in range(epochs):
    resnet_ft.train()
    running_loss = 0

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        outputs = resnet_ft(images)
        loss = criterion(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    print(f"ResNet50 FT Epoch {epoch+1}: {running_loss/len(train_loader):.4f}")

ResNet50 FT Epoch 1: 0.4033
ResNet50 FT Epoch 2: 0.2350
ResNet50 FT Epoch 3: 0.1879
ResNet50 FT Epoch 4: 0.1912
ResNet50 FT Epoch 5: 0.1941
```

Fine-Tuning (VGG16)

```
[32]
✓ 1s
vgg16_ft = models.vgg16(pretrained=True)

for param in vgg16_ft.parameters():
    param.requires_grad = False

# Unfreeze last conv layers
for param in vgg16_ft.features[-5:].parameters():
    param.requires_grad = True

vgg16_ft.classifier[6] = nn.Linear(4096, num_classes)

vgg16_ft = vgg16_ft.to(device)
```

Train Fine-Tuned VGG16

```
[33] 1m ▶ optimizer = optim.Adam(filter(lambda p: p.requires_grad, vgg16_ft.parameters()), lr=0.0001)

for epoch in range(epochs):
    vgg16_ft.train()
    running_loss = 0

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        outputs = vgg16_ft(images)
        loss = criterion(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    print(f"VGG16 FT Epoch {epoch+1}: {running_loss/len(train_loader):.4f}")
```

```
... VGG16 FT Epoch 1: 0.4003
VGG16 FT Epoch 2: 0.3238
VGG16 FT Epoch 3: 0.2813
VGG16 FT Epoch 4: 0.2697
VGG16 FT Epoch 5: 0.2521
```

Evaluation

```
[34] 49s ▶ def evaluate(model):
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)

            outputs = model(images)
            _, preds = torch.max(outputs, 1)

            correct += (preds == labels).sum().item()
            total += labels.size(0)

    return 100 * correct / total

print("VGG16 FE:", evaluate(vgg16))
print("ResNet50 FE:", evaluate(resnet_fe))
print("ResNet50 FT:", evaluate(resnet_ft))
print("VGG16 FT:", evaluate(vgg16_ft))
```

```
... VGG16 FE: 89.85507246376811
ResNet50 FE: 90.94202898550725
ResNet50 FT: 91.66666666666667
VGG16 FT: 90.94202898550725
```